

Vers une formalisation adaptée à la détection d'anomalie multi-domaines

Contents

3.1	Objectifs et cadre de détection	2
3.2	État de l'art de la détection d'anomalies	3
3.2.1	Approche historique dans les réseaux traditionnels	3
3.2.2	Apprentissage profond appliqué aux réseaux 5G	4
3.2.3	Comparaison aux techniques des systèmes distribués	7
3.2.4	Conclusion	8
3.3	Méthodologie de formalisation	8
3.3.1	Extraction des features depuis la payload applicative	9
3.3.2	Transformation en graph	9
3.3.3	Conclusion	12
3.4	Détection d'anomalies dans des graphs	12
3.4.1	Introduction aux GNN	12
3.4.2	Etat de l'art des GNN	14
3.4.3	Formalisation adapté à l'utilisation de GNN dynamiques . .	18
3.4.4	Fonctionnement détaillé des GNN dynamique	19
3.4.5	Adaptation à notre cas d'utilisation	22
3.5	Encodage des paramètres de dimension non-fini (NLP) . .	25
3.5.1	Sélection et encodage des features	25
3.6	Conclusion	25

Le passage aux réseaux 5G s'accompagne d'un changement de paradigme majeur, caractérisé par une architecture distribuée et une complexification des interactions entre entités. Par ailleurs, l'émergence d'anomalies sémantique, séquentielle et topologique impose de nouvelles exigences en matière d'analyse. Dans ce contexte, il devient nécessaire non seulement d'adopter une analyse fine au niveau des paquets, mais également de prendre en compte le contexte dans lequel s'inscrivent les échanges. Or, cette problématique reste relativement récente

dans le domaine des télécommunications, et les méthodes actuelles de détection d'anomalies apparaissent inadaptées à ce nouveau paradigme.

Dans ce contexte, nous proposons de représenter les échanges au sein des réseaux 5G sous la forme d'un graphe dynamique, permettant de capturer à la fois les interactions entre les différentes entités du réseau ainsi que l'évolution de ces interactions dans le temps. Cette formalisation nous permet d'exploiter des techniques issues du domaine des GNN intrinsèquement adaptées à l'analyse des interactions complexes. Nous nous intéressons plus particulièrement aux GNN dynamiques, qui intègrent nativement la dimension temporelle.

3.1 Objectifs et cadre de détection

Dans ces travaux, nous nous plaçons du point de vue de l'opérateur d'un réseau 5G et nous nous intéressons à la détection d'anomalies en nous concentrant exclusivement sur le trafic de contrôle échangé entre les entités du réseau, qu'il s'agisse du cœur de réseau ou du RAN. Le trafic utilisateur n'est pas pris en compte, car il ne présente pas de spécificités propres à l'architecture 5G et ne relève pas de la responsabilité directe du fournisseur d'accès réseau.

En se plaçant du point de vue de l'opérateur, nous supposons que celui-ci possède l'ensemble des NF et dispose d'un accès complet au trafic de contrôle. Dans l'architecture 5G, la fonction Service Communication Proxy (SCP) agit comme un intermédiaire dans les échanges entre les différentes NF, notamment pour le routage des requêtes et la vérification des autorisations d'accès. Dans ce contexte, il est raisonnable de considérer que l'opérateur peut observer l'intégralité du trafic de contrôle en déployant des sondes au niveau des SCP, qui constituent des points d'observation privilégiés des interactions entre NF.

Ce trafic est généralement chiffré, par exemple via TLS sur le SBI ou via IPsec pour certains canaux, tels que N3 ou N4, reposant sur des protocoles spécifiques à la 5G. Cependant, en adoptant le point de vue de l'opérateur, nous supposons disposer des clés de chiffrement nécessaires pour accéder au contenu de ces échanges en clair.

L'objectif serait de pouvoir exécuter nos modèles de détection en temps réel. Malgré les débits élevés offerts par la 5G, le volume de données est majoritairement porté par le trafic utilisateur, qui n'entre pas dans le périmètre de cette étude. À l'inverse, le trafic de contrôle est principalement composé de messages liés aux changements d'état des entités du réseau, qu'il s'agisse d'UE ou de NF, et ont une fréquence d'occurrence plus modérée. Par conséquent, bien que le surcoût induit par nos modèles puisse avoir un impact sur les performances, il est raisonnable de supposer que les ressources disponibles sont suffisantes pour permettre un traitement en temps réel, notamment grâce à l'exploitation de techniques d'optimisation et de parallélisation.

La remédiation des anomalies détectées ne relève pas du périmètre de ce travail et est considérée comme une brique fonctionnelle distincte. Nous nous concentrons

exclusivement sur la détection et les alertes générées par notre système ont vocation à être exploitées de manière similaire à celles produites par un système de détection d'intrusion (IDS) classique. Elles peuvent ainsi conduire, par exemple, à des ajustements des règles de firewall ou à des modifications de l'architecture réseau.

L'objectif de ces travaux n'est pas de concevoir un système de détection absolu et entièrement autonome. En effet, une anomalie ne constitue pas nécessairement le signe d'une activité malveillante et peut résulter d'une mise à jour, d'une erreur de configuration ou encore d'une interconnexion avec un système tiers, comme dans le cas du roaming. Dans ce contexte, la distinction entre une anomalie bénigne et une attaque de type zero-day requiert in fine une expertise humaine. Par conséquent, l'un des aspects essentiels de nos travaux est de proposer un système interprétable, venant compléter une expertise humaine plutôt que de la remplacer.

3.2 État de l'art de la détection d'anomalies

Cette section propose un panorama des travaux de recherche consacrés à la détection d'anomalies. Dans un premier temps, nous présenterons les limites des approches traditionnelles appliquées à notre problématique, en montrant que les objectifs visés, les contraintes opérationnelles ainsi que les zones d'observation diffèrent sensiblement de notre contexte d'étude. Nous nous intéresserons ensuite aux méthodes développées pour la détection d'anomalies dans les réseaux 5G, afin d'évaluer l'état actuel de la recherche sur notre problématique spécifique. Nous pourrions ainsi identifier les limites ainsi que les perspectives encore peu explorées. Enfin, nous adopterons une perspective plus large en examinant les travaux issus du domaine des systèmes distribués qui, comme nous l'avons montré précédemment, présentent de nombreuses similarités structurelles et fonctionnelles avec les architectures 5G.

3.2.1 Approche historique dans les réseaux traditionnels

La détection d'attaques au sein des réseaux est historiquement assurée par des systèmes de détection d'intrusion (IDS). Ces systèmes reposent généralement sur une analyse du trafic agrégé sous forme de flux, plutôt qu'au niveau des paquets individuels. Cette approche permet d'extraire des métriques globales et d'effectuer des analyses de volumétrie, par exemple afin de détecter des variations significatives du volume de trafic ou repérer des signatures d'attaques connues.

Parmi les IDS les plus connus on retrouve Snort [?], Zeek [?] et Suricata [?], qui reposent sur des règles prédéfinies pour identifier des motifs (aussi appelés signatures) d'attaques spécifiques dans le flux réseau. Ce sont la plupart du temps des outils open source qui bénéficient d'une large communauté qui contribue à l'enrichissement de leur base de signatures.

Par ailleurs, des approches fondées sur l'apprentissage automatique supervisé ont également été proposées pour la détection d'intrusion, notamment à l'aide de réseaux bayésiens [?], de machines à vecteurs de support (SVM) [?] ou encore

d'arbres de décision [?]. Ces méthodes présentent l'avantage d'être relativement transparentes, interprétables et efficaces pour détecter des attaques déjà connues.

Parmi les approches non supervisées, Münz et al. [?] proposent, par exemple, de découvrir des motifs anormaux en adaptant l'algorithme K-Means au trafic réseau, permettant ainsi de regrouper les données en fonction de leur similarité sans nécessiter d'étiquettes. Néanmoins, les caractéristiques utilisées en entrée restent relativement limitées et se restreignent notamment au nombre de paquets, au volume d'octets ainsi qu'aux adresses source et destination. Une telle approche permet d'identifier des motifs liés à la volumétrie et, par conséquent, de détecter certains types d'attaques par déni de service (DoS). Toutefois, comme elle ne prend en compte ni le contenu sémantique des paquets ni le contexte des échanges, elle demeure insuffisante pour détecter des attaques plus subtiles.

Ariu et al. [?] proposent quant à eux d'utiliser des modèles de Markov cachés afin de modéliser le trafic réseau sous forme de séquences d'octets. Ces modèles apprennent des probabilités de transition à partir de trafic bénin, puis évaluent dans quelle mesure un trafic observé s'en écarte. Cependant, les séquences considérées sont limitées à des fenêtres de taille relativement réduite, ce qui empêche la capture de dépendances temporelles à long terme. De plus, cette approche ne permet pas de modéliser explicitement les interactions entre paquets.

Cette étude met en évidence le décalage entre les approches historiques de détection d'intrusion et les besoins spécifiques liés aux réseaux 5G. Les méthodes traditionnelles, principalement conçues pour l'analyse agrégée de flux et la détection de signatures connues, se révèlent peu adaptées à un environnement distribué, dynamique et fortement évolutif. Les approches non supervisées constituent une piste plus pertinente, car elles permettent de détecter des comportements anormaux sans connaissance préalable des attaques. Toutefois, les travaux existants dans les IDS traditionnels restent généralement limités, soit par la faible richesse des caractéristiques considérées, soit par une modélisation insuffisante des dépendances temporelles et des interactions complexes entre paquets. Ces limites justifient ainsi la nécessité de développer des méthodes plus adaptées aux spécificités des architectures 5G.

3.2.2 Apprentissage profond appliqué aux réseaux 5G

Après avoir présenté les limites des approches traditionnelles de détection d'intrusion, nous nous intéressons désormais aux méthodes spécifiquement conçues pour les réseaux 5G. L'objectif est d'analyser dans quelle mesure ces travaux prennent en compte les dimensions sémantique, séquentielle et topologique. Cette analyse permettra d'identifier les avancées récentes du domaine, mais également les limitations persistantes des approches existantes.

Parmi les travaux de détection spécifiques à la 5G, Kale et al. [?] proposent d'utiliser des réseaux de neurones convolutionnels (CNN) en considérant les octets d'un paquet comme une image sur laquelle sont ensuite étudiées des variations structurelles. Les modèles CNN obtiennent de très bonnes performances dans de

nombreux domaines, y compris dans des tâches qui ne relèvent pas intrinsèquement de la vision par ordinateur. Toutefois, comme ils n'interprètent pas le contenu des données, leur capacité de détection se limite à un sous-ensemble d'attaques sémantiques dans lesquelles les valeurs s'écartent fortement du trafic normal. Ils ne permettent donc pas de détecter des variations légères de valeurs d'identifiants comme les adresses IP ou les IMSI. De plus, le travail présenté par Kale et al. se concentre uniquement sur la détection sémantique et ne prend pas en compte le contexte des échanges. Lam et al. [?] ajoutent un contexte temporel à leur CNN en alimentant ses sorties dans un réseau de neurones récurrent RNN. Cependant, en plus de ne pas être satisfaisant sur le domaine sémantique, leur modèle ne prend pas en compte l'identité des acteurs impliqués dans les échanges et n'est donc pas capable de détecter des attaques relationnelles.

AutoGuard [?] et ADSeq-5GCN [?] modélisent les échanges réseau sous forme de séquences d'interactions entre entités du réseau et utilisent également des RNN afin de capturer les dépendances temporelles. Leurs approches intègrent l'identité des entités communicantes, ce qui leur permet de prendre en compte, dans une certaine mesure, le contexte topologique des interactions à court terme. Cependant, elles rencontrent des difficultés à modéliser efficacement les dépendances relationnelles lorsque les interactions sont éloignées dans le temps. De plus, le principal problème est que leur formalisme n'intègre aucunement le contenu des paquets et ne considère donc aucunement la dimension sémantique. 5GCGuard [?] tente de pallier cette limitation en enrichissant leurs séquences par l'intégration de l'URL associée à chaque message. Bien qu'elle apporte une information sémantique supplémentaire, cette approche demeure très limitée car l'information propre à l'URL ne représente qu'une fraction du contenu réel des paquets et n'est applicables qu'au trafic HTTP.

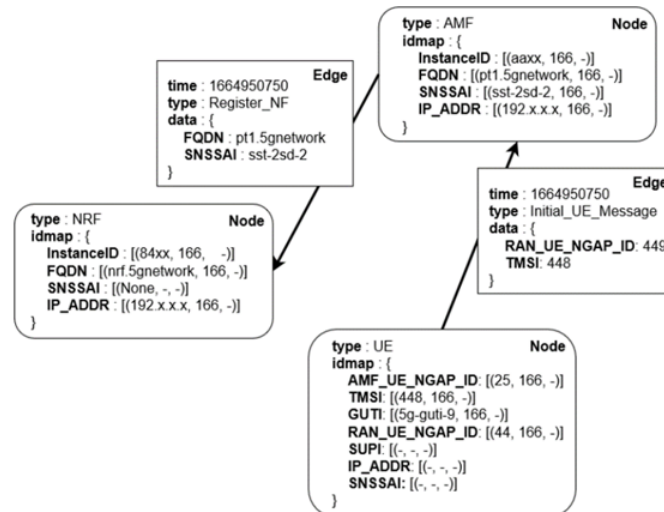


Figure 3.1: Trafic de contrôle 5G formalisé par Prov5GC [?] en tant que graphe de provenance

Prov5GCC [?] propose de son côté de représenter le trafic sous forme de graphe de provenance. Dans cette représentation, illustrée sur la figure ??, les nœuds correspondent aux entités du réseau et chaque message échangé entre ces entités est modélisé par une arête, à laquelle est associé le contenu du paquet. La formalisation en graph est particulièrement adaptée à l'analyse topologique des interactions du réseau. Cependant, elle ne modélise pas explicitement la dimension temporelle des échanges, ce qui restreint la capacité à capturer l'évolution dynamique des interactions. De plus, le fait d'agréger le payload au niveau des arêtes limite fortement l'exploitation de l'information contenue dans les paquets. Dans leur cas la payload est utilisé par un algorithme de détection heuristique spécifique à chaque attaque, ce qui rend l'approche relativement lourde et peu adaptée à des environnements complexes.

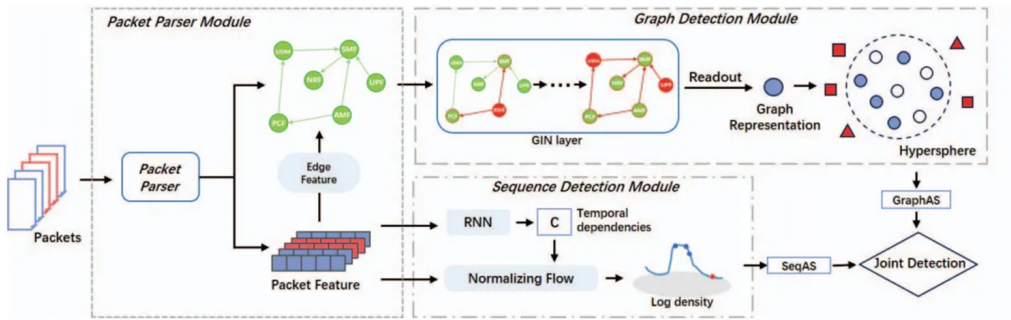


Figure 3.2: Pipeline de GSAD [?] représentant leur formalisation ainsi que les trois modules de détections traitant respectivement les dimensions sémantique, séquentielle et topologique

GSAD [?] propose également de modéliser les données sous forme de graphe de manière similaire à Prov5GC [?]. La principale différence réside dans la représentation du contenu des arêtes. Au lieu de l'encoder sous forme de dictionnaire monolithique, les auteurs le transforment en flux d'octets structuré comme une image. Cette représentation leur permet d'utiliser un CNN pour encoder chaque paquet et analyser sa dimension sémantique. Les représentations sémantiques sont ensuite injectées dans un GNN afin de modéliser les interactions entre les paquets, puis dans un RNN pour capturer les dépendances temporelles. À notre connaissance, GSAD [?] est la seule approche de détection d'anomalies 5G basée sur l'apprentissage profond qui considère simultanément les trois dimensions de détection. La principale limite de leur approche réside dans le fait que, comme évoqué précédemment, les CNN ne permettent qu'une analyse relativement superficielle de la dimension sémantique des paquets. La Figure ?? illustre l'architecture générale du pipeline proposé par GSAD et met en évidence les trois composantes du modèle correspondant respectivement aux trois dimensions.

Le tableau ?? résume les différentes approches de détection d'anomalies dans les réseaux 5G, en mettant en évidence les dimensions sémantique, temporelle et

Référence	Sémantique	Séquentiel	Topologique
Kale et al. [?]	Oui	Non	Non
Lam et al. [?]	Oui	Oui	Non
AutoGuard [?]	Non	Oui	Partiel
ADSeq-5GCN [?]	Non	Oui	Partiel
5GCCGuard [?]	Non	Oui	Partiel
Prov5GC [?]	Partiel	Non	Partiel
GSAD [?]	Partiel	Partiel	Oui

Table 3.1: Comparaison des approches de l’état de l’art de détection d’anomalies 5G selon les dimensions considérées

topologique prises en compte par chacune d’elles. L’état de l’art montre que peu d’approches considèrent simultanément les trois dimensions. Les approches basées sur les GNN apparaissent particulièrement adaptées pour modéliser les interactions entre entités. À l’inverse, aucune des modélisations proposées ne semble réellement adaptée au traitement du contenu sémantique des paquets, celui-ci étant le plus souvent considéré comme un bloc monolithique.

3.2.3 Comparaison aux techniques des systèmes distribués

Enfin, nous prenons un peu de recul en nous intéressant aux travaux issus du domaine des systèmes distribués. Comme évoqué précédemment, les architectures 5G changent de paradigme et présentent maintenant de nombreuses similarités avec ces systèmes fortement distribués et dynamiques. Nous examinons donc dans cette section les principales approches de détection d’anomalies proposées dans ce domaine, afin d’identifier dans quelle mesure elles exploitent les spécificités de ces architectures et les enseignements qu’il est possible d’en tirer pour notre problématique.

Parmi les approches exploitant la nature distribuée des infrastructures, on retrouve notamment des travaux basés sur le Federated Learning (FL). DioT[?] et VFL[?], par exemple, mettent en avant les avantages de la fédération dans les systèmes distribués, notamment en termes de passage à l’échelle, de confidentialité et de décentralisation de l’apprentissage. Cependant, leur contribution se concentre principalement sur le mécanisme de fédération lui-même. Les modèles utilisés restent relativement classiques et ne couvrent généralement qu’une partie des dimensions nécessaires à une détection complète des anomalies. DioT[?] s’appuie ainsi sur des GRU pour modéliser les dépendances temporelles, tandis que VFL[?] exploite GraphSAGE afin de capturer les relations structurelles entre entités. Dans ce contexte, le Federated Learning apparaît davantage comme une extension permettant de rendre la détection plus scalable et plus réaliste d’un point de vue opérationnel, plutôt que comme une approche introduisant de nouveaux mécanismes de détection fondamentalement différents.

Dans la littérature des systèmes distribués, on retrouve également de nombreuses

approches fondées sur l'analyse de logs systèmes, telles que [?, ?]. Toutefois, ces méthodes présentent plusieurs limitations. Bien qu'elles permettent d'observer l'activité du système à un niveau relativement élevé, les logs constituent une représentation indirecte et partielle des événements réseau. Ils dépendent fortement des implémentations logicielles et peuvent contenir du bruit, des informations manquantes ou des incohérences. Ainsi, certains comportements anormaux peuvent ne jamais être journalisés, ce qui limite intrinsèquement la capacité de détection. Une piste complémentaire consiste à combiner différentes sources d'observation, comme proposé par Bogatinovski et al.[?], en fusionnant logs et traces afin d'enrichir la représentation des événements. Cependant, cette approche introduit également une complexité supplémentaire dans la gestion et la corrélation des données, et peut conduire à une surcharge informationnelle. Dans notre cas, les paquets réseau fournissent déjà une observation suffisamment riche et fine du système. Ainsi, l'ajout de sources supplémentaires risquerait alors davantage de brouiller l'information pertinente que de réellement améliorer la détection.

Enfin, une part importante des travaux de la littérature s'intéresse aussi aux GNN [?, ?] et plus spécifiquement sur des approches dynamiques[?]. Cette convergence confirme que la modélisation par graphes constitue un cadre particulièrement pertinent pour capturer les interactions complexes au sein des systèmes distribués, en intégrant à la fois les dimensions structurelles et temporelles.

3.2.4 Conclusion

L'analyse de la littérature sur la détection d'anomalies réseau met en évidence que les approches traditionnelles ne sont pas adaptées à notre problématique, principalement en raison d'un niveau de granularité trop élevé qui empêche la capture de comportements subtils. En se focalisant sur des travaux spécifiquement dédiés à la 5G, on constate par ailleurs qu'aucune approche ne propose à ce jour un formalisme unifié permettant de prendre en compte simultanément les trois dimensions de détection définies dans ce travail. Néanmoins, plusieurs éléments issus de l'état de l'art convergent vers des directions prometteuses. En particulier, des travaux tels que GSAD, ainsi que les approches issues des systèmes distribués, montrent que les représentations par graphes et les GNN font désormais l'objet d'un consensus croissant. Ces modèles apparaissent comme des outils particulièrement adaptés pour représenter la structure complexe des interactions et pourraient constituer une base solide pour répondre à notre problématique.

3.3 Méthodologie de formalisation

Nous avons vu que le formalisme en graphe proposé par GSAD consiste à regrouper l'ensemble du contenu des paquets dans un bloc monolithique associé à une arête, ce qui limite fortement la capacité à exploiter correctement la dimension sémantique. Dans ce contexte, nous proposons une représentation plus fine du trafic, en décomposant le contenu des paquets en nœuds indépendants correspondant

chacun à un paramètre différent. Nous détaillons à présent notre méthodologie de formalisation. Nous commencerons par l'extraction des caractéristiques à partir du trafic de contrôle, puis nous présenterons leur transformation en graphes.

3.3.1 Extraction des features depuis la payload applicative

Comme notre objectif est principalement d'analyser le contenu applicatif des paquets, nous extrayons la payload ainsi que l'ensemble des champs disponibles pour chaque paquet. Les seules informations non applicatives conservées concernent la couche IP, utilisées comme identifiant afin de discriminer les différentes entités du réseau. Cette approche n'est pas idéale, car elle est sensible au *spoofing*. Toutefois, dans un cadre opérationnel, ces identifiants pourraient être remplacés par des mécanismes plus robustes, tels que des empreintes ou des hash représentant les entités de manière unique.

Pour récupérer le contenu des payloads, nous utilisons *Scapy*, qui permet de parser les paquets réseau et de générer, pour chacun d'eux, une représentation sous forme de dictionnaire associant les noms des paramètres à leurs valeurs respectives.

Une des problématiques du trafic applicatif est la présence d'imbrication dans le contenu des données de la payload. En effet, les paquets HTTP/2 contiennent couramment des structures JSON avec des données hiérarchisées. Conserver explicitement le chemin d'imbrication dans une modélisation en graph nécessiterait d'introduire de nouveaux types de noeuds et donc de diluer l'information produite par le contenu lui même. Or, même si l'imbrication est utile pour différencier des paramètres, elle reste secondaire par rapport aux valeurs elles-mêmes.

Pour gérer ce problème, nous avons donc choisi d'aplatir les dictionnaires extraits des payloads en construisant le nom de chaque paramètre de feuille comme la concaténation de ses clés le précédentes, séparées par des délimiteurs. On obtient ainsi des identifiants de paramètres composés, par exemple `http2.headers.content-type`, permettant de désigner le champ `content-type` d'un paquet HTTP/2. Cette approche permet de conserver l'information d'imbrication en tant que nom de paramètre tout en maintenant une structure suffisamment simple pour notre formalisation en graphe.

3.3.2 Transformation en graph

Au lieu de représenter un paquet comme une arête entre deux entités, nous le modélisons comme un sous-graphe, où chaque paramètre est un nœud. Chacun de ces nœuds paramètre est relié à un nœud central qui représente le paquet. Chaque nœud paramètre contient la valeur de ce dernier, tandis que l'arête reliant ce nœud au à celui central porte le nom du paramètre correspondant. Lors de la création d'un sous-graphe représentant un paquet, de nouveaux nœud paramètres sont créés seulement s'il n'existaient pas encore dans le graph global. Dans le cas contraire, le nœud existant est réutilisé. Ce mécanisme permet de relier progressivement les différents sous-graphes entre eux à travers leurs paramètres partagés. La figure ??

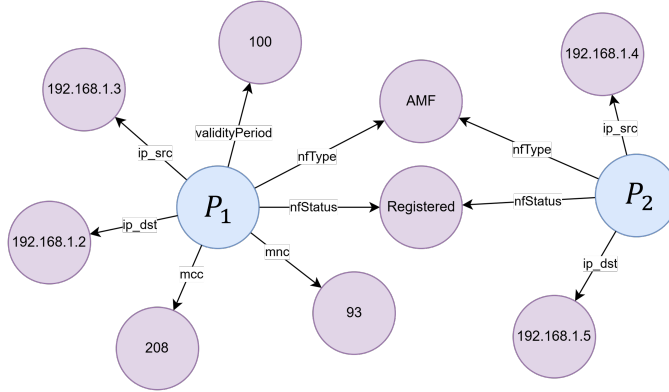


Figure 3.3: Exemple de transformation de paquets en sous-graphes. Les nœuds mauves représentent les valeurs des paramètres, tandis que les nœuds bleus représentent les paquets eux-mêmes.

illustre le processus de transformation de deux paquets en deux sous-graphes. Ces deux paquets partagent certains paramètres ayant des valeurs communes et leurs sous-graphes respectifs se retrouvent donc connectés.

Dans notre formalisme nous avons donc deux types de noeuds, paramètres et centraux qui ne peuvent jamais être reliés directement à d'autres noeuds du même type, donc un graphe biparti. Notre graphe est par ailleurs purement additif. La réception d'un paquet ne peut entraîner que l'ajout de nouveaux nœuds et d'arêtes, sans jamais modifier ni supprimer ceux déjà présents.

Cette décomposition permet de connecter des attributs issus de paquets différents en s'appuyant sur des caractéristiques communes, plutôt que sur la seule structure source-destination. Une telle représentation offre un niveau de granularité très fin et permet de capturer des interactions plus riches entre les caractéristiques des paquets. De plus, cette approche permet également d'identifier précisément les paramètres à l'origine d'un comportement anormal et se révèle donc, par construction, hautement interprétable.

Nous allons maintenant analyser la représentation des anomalies de chaque dimension une fois adaptés à notre formalisme.

3.3.2.1 Anomalies sémantiques

La figure ?? illustre une anomalie sémantique dans laquelle un champ inhabituel apparaît. Dans le cas présent, il s'agit du paquet P3 qui contient une valeur de son champs *nf-type* qui n'existe pas en 5G. Cette anomalie peut être détectée en le comparant aux autres paquets de même nature. On observe par exemple que les paquets P1 et P2, qui sont, comme P3, des requêtes vers l'endpoint *add-nf*, possèdent des valeurs identiques pour le champ *nf-type*, alors que P3 s'en écarte. Un modèle de détection ayant appris la structure usuelle de ce type de paquet pourrait ainsi identifier l'anomalie en observant uniquement le sous-graphe associé

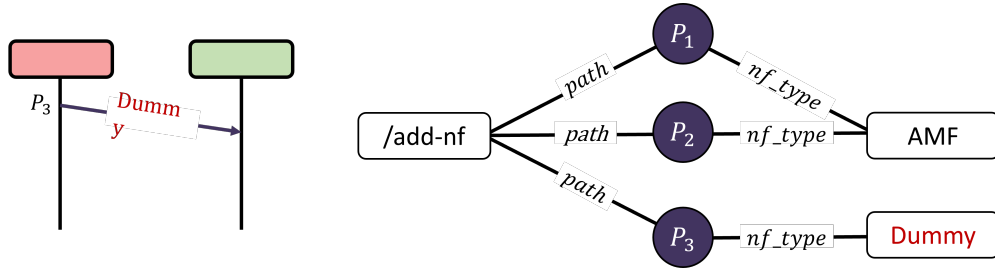


Figure 3.4: Anomalie sémantique et sa représentation sous forme de graphe

à P3.

3.3.2.2 Anomalies topologiques

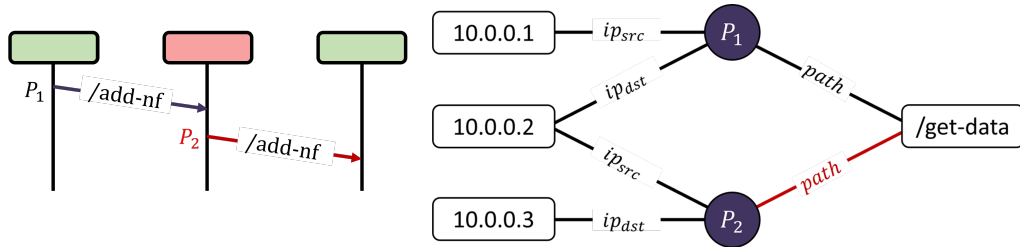


Figure 3.5: Anomalie topologique et sa représentation sous forme de graphe

Sur la figure ??, on observe un exemple d'attaque de type man-in-the-middle, où un paquet P1, initialement envoyé à une première adresse, génère l'envoi d'un nouveau paquet P2 identique (ici simplifié par le fait qu'il s'agisse du même endpoint). Ce nouveau paquet a comme source la destination du paquet précédent et comme destination une nouvelle adresse. Ce cas ne peut normalement pas survenir en 5G, car il n'existe aucun scénario légitime de redirection de trafic de ce type. Le paquet P2 est une copie de P1 et paraît donc légitime pris individuellement. Il est nécessaire de prendre en compte le contexte de redirection afin de détecter l'anomalie.

3.3.2.3 Anomalies séquentielles

Enfin, sur la figure ??, on observe un exemple d'anomalie séquentielle. Dans ce cas, le contenu et les interactions des paquets sont normaux et la caractéristique anormale concerne uniquement leur ordre d'arrivée. Le problème est que le formalisme actuel ne modélise pas explicitement la dimension temporelle. Il serait possible de l'intégrer en ajoutant à chaque sous-graphe un nœud représentant le temps d'observation du paquet, mais avec cette approche la dimension temporelle serait confondue avec les autres paramètres, alors qu'elle est de nature distincte et particulièrement importante. Il est donc nécessaire de proposer une modélisation permettant d'intégrer le temps de manière plus explicite.

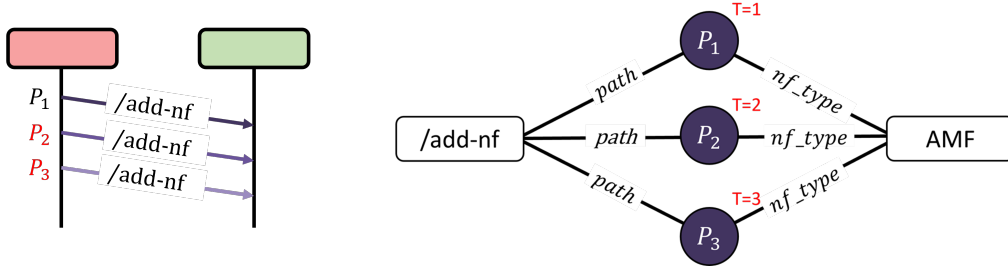


Figure 3.6: Anomalie séquentielle et sa représentation sous forme de graphe

3.3.3 Conclusion

Notre formalisme se distingue par sa capacité à mieux appréhender le contenu sémantique, un aspect souvent négligé dans l'état de l'art. Cependant, notre modélisation actuelle demeure statique et ne prend pas en compte la dimension temporelle. Dans le but de surmonter cette limitation, nous explorons les graphes dynamiques, qui intègrent nativement cette dimension temporelle, et dont nous présenterons les principes fondamentaux dans les sections suivantes.

3.4 Détection d'anomalies dans des graphes

La détection d'anomalies à l'aide de GNN constitue un domaine de recherche en pleine expansion, connaissant un développement significatif ces quinze dernières années. Nous introduisons ici les principes fondamentaux et les caractéristiques essentielles des GNN. Nous examinerons ensuite l'état de l'art des GNN puis. Enfin, nous verrons comment adapter notre formalisme présenté précédemment aux graphes dynamiques.

3.4.1 Introduction aux GNN

Le fonctionnement des GNN est très proches de celui des CNN présentés précédemment, la principale différence étant le formalisme. Dans les deux cas, le principe repose sur une agrégation locale de l'information. Dans une image, cette agrégation s'effectue sur des voisinages de pixels, tandis que dans un graphe elle s'applique au voisinage des nœuds. Certains travaux proposent notamment de considérer une image comme un graphe, où chaque pixel est un nœud relié à ses voisins par un arc, et d'y appliquer des GNN. Néanmoins, cette approche reste relativement marginale [?].

L'idée fondamentale de ces deux types de modèles est que chaque itération du processus de convolution, appelé *Message Passing* pour les GNN, permet de diffuser l'information aux voisins directs. En procédant de manière récursive, chaque couche de convolution propage cette information à une distance croissante, augmentant progressivement le champ de réception des représentations. La figure ?? illustre ce processus de diffusion de l'information réalisé par deux convolutions successives.



Figure 3.7: Illustration du processus de convolution. La première figure représente le graphe à son état initial. Dans la deuxième, une convolution est appliquée : on observe que les informations des nœuds feuilles sont agrégées et propagées à leurs voisins directs. Dans la troisième, une deuxième convolution est réalisée, et les informations précédemment propagées sont à nouveau diffusées jusqu'au nœud central du graphe, qui contient ainsi une représentation de son voisinage complet à distance 2.

En apprentissage sur graphes, on distingue généralement les approches transductives et inductives selon les informations disponibles au moment de l'entraînement. En mode transductif, le modèle connaît déjà le graphe complet, y compris les nœuds à prédire et leur voisinage, même si leurs labels restent inconnus. À l'inverse, l'apprentissage inductif suppose que les nœuds à traiter pendant le test ne sont pas visibles durant l'entraînement. Le modèle doit donc apprendre des règles générales capables de se transférer à de nouvelles structures. Le choix entre ces deux paradigmes dépend fortement de la tâche considérée. Dans certains cas, le cadre transductif est plus adapté, tandis que dans d'autres, notamment lorsque la généralisation à de nouveaux nœuds est nécessaire, l'approche inductive est préférable.

De par leur nature évolutive, l'apprentissage inductif est souvent privilégié dans les travaux sur les graphes dynamiques, car il permet de généraliser à des nœuds et interactions apparaissant au cours du temps. Néanmoins, comme le soulignent Longa et al.[?], un GNN entraîné en mode transductif pour la classification de nœuds peut tout de même être appliqué à un graphe inconnu, ce qui revient à effectuer de l'apprentissage inductif. Bien que l'apprentissage inductif paraisse le plus adapté à notre problématique dynamique, nous considérerons donc tout de même aussi l'apprentissage transductif.

Parmi les tâches courantes dans l'état de l'art des graphes, on retrouve principalement la classification, la régression et le clustering. Ces tâches peuvent être appliquées à différents niveaux de granularité, que ce soit au sur des nœuds, des arêtes ou des graphes entiers. La classification consiste à attribuer un label discret à un élément du graphe, tandis que la régression vise à prédire une valeur continue associée à celui-ci. Le *clustering*, quant à lui, a pour objectif de regrouper les éléments du graphe selon leur similarité structurelle ou leurs représentations apprises. On retrouve également la tâche de *link prediction*, qui consiste à prédire l'apparition future d'arêtes entre des nœuds, ou plus généralement à estimer la

probabilité d'existence d'une connexion entre deux entités du graphe. Cette tâche est particulièrement importante dans les graphes dynamiques, où l'objectif est d'anticiper l'évolution des interactions au cours du temps. C'est cette dernière famille de tâches qui nous intéresse particulièrement, puisque notre objectif est d'estimer, pour chaque valeur de paramètre, la probabilité qu'elle apparaisse et qu'elle soit donc considérée comme légitime.

L'étendue des problèmes pouvant être modélisés sous forme de graphes, ainsi que la diversité des cas d'utilisation potentiels, expliquent la croissance rapide de la littérature sur ce sujet.

3.4.2 Etat de l'art des GNN

Nous analyserons désormais l'état de l'art concernant les GNN. Notre étude s'attachera à l'évolution historique de ces approches, en commençant par les GNN statiques. Nous examinerons ensuite les graphes dynamiques et les différentes architectures de GNN adaptées à la manipulation de ces types de graphes. Enfin, nous analyserons les approches non supervisées, indépendamment de la dimension temporelle, afin d'évaluer la capacité des GNN à traiter des données non étiquetées.

3.4.2.1 GNN traditionnels

Le domaine des GNN est relativement récent et s'est fortement développé au cours de la dernière décennie. Le concept de réseaux de neurones sur graphes a été formellement introduit pour la première fois en 2008 par Scarselli et al. [?], qui ont proposé un framework pour l'apprentissage sur des données structurées en graphe, basé sur des mécanismes itératifs de propagation et d'agrégation d'états. Des travaux ultérieurs ont considérablement amélioré la praticabilité et la scalabilité de cette formulation initiale. En particulier, Kipf et Welling [?] ont introduit les Graph Convolutional Networks (GCN), qui simplifient le processus de propagation des messages en linéarisant l'agrégation de voisinage.

Dans leur formulation moderne, les GNN peuvent être vus comme réalisant des convolutions basées sur le voisinage, où l'information provenant des nœuds adjacents est agrégée afin d'enrichir la représentation de chaque nœud avec un contexte local. La plupart des architectures contemporaines de GNN suivent le paradigme de *message passing* proposé par Gilmer et al. [?], composé de trois phases principales : la construction des messages à partir des embeddings des nœuds, l'agrégation des messages provenant des voisins, et la propagation (ou mise à jour) de l'information agrégée pour produire de nouvelles représentations des nœuds.

Sur la base du cadre MPNN standard, plusieurs extensions ont été proposées. Par exemple, les Graph Attention Networks (GAT) [?] introduisent des mécanismes d'attention qui attribuent des poids adaptatifs aux nœuds voisins lors de l'agrégation. GraphSAGE [?] permet un apprentissage inductif sur des nœuds jusqu'alors non vus en échantillonnant un voisinage de taille fixe et en appliquant des fonctions d'agrégation apprenables telles que la moyenne, le pooling ou des

agrégateurs basés sur LSTM. D'autres variantes notables incluent les Graph Isomorphism Networks (GIN) [?], qui atteignent un pouvoir discriminant maximal parmi les GNN basés sur l'agrégation de voisinage.

Cependant, la plupart de ces architectures de GNN sont principalement conçues pour des graphes statiques. Or, nos données évoluent dans le temps et le graphe sous-jacent est dynamique. Nous nous intéressons donc aux modèles de graphes dynamiques, une classe encore plus récente de GNN. Parmi ces modèles, on distingue deux grandes catégories selon la granularité temporelle.

3.4.2.2 GNN Dynamiques

La première famille de graphes dynamiques, les réseaux de graphes à temps discret (DTGN), constitue l'approche la plus directe pour intégrer l'information temporelle. Dans ce cadre, l'évolution du graphe est modélisée comme une suite de snapshots observés à intervalles réguliers. La dimension temporelle est ensuite prise en compte via des modèles récurrents appliqués en complément du GNN. C'est notamment le cas de T-GCN [?] et EvolveGCN [?]. Cette formulation permet d'adapter relativement facilement des architectures de GNN classiques à un contexte temporel. DySAT [?] va un cran plus loin et propose un mécanisme d'attention temporelle : pour chaque nœud, le modèle exploite une séquence de ses représentations historiques afin de capturer l'évolution de son contexte au cours du temps. Bien que ces approches soient relativement simples à mettre en œuvre, elles présentent plusieurs limites. En particulier, le fait de reposer sur une représentation par snapshots implique de conserver l'état complet du graphe à plusieurs instants, ce qui engendre un coût mémoire et calculatoire élevé. De plus, ces méthodes peinent à capturer des dépendances temporelles longues, notamment lorsque les interactions sont irrégulières ou très espacées dans le temps.

La seconde catégorie correspond aux réseaux de graphes en temps continu, qui opèrent à une granularité temporelle plus fine que les approches discrètes. Plutôt que de reposer sur des instantanés fixes, ces modèles traitent des séquences d'événements correspondant à l'ajout, la suppression ou la modification de nœuds et d'arêtes dans le graphe. Cette modélisation au niveau événementiel permet d'intégrer directement les dépendances temporelles dans le modèle, plutôt que de les traiter de manière externe.

Parmi les modèles de graphes dynamiques en temps continu, on distingue généralement deux sous-familles : les modèles d'encodage temporel *Temporal Embedding* et ceux de de voisinage temporel *Temporal Neighborhood*. Les modèles *Temporal Embedding* ne stockent pas explicitement d'état associé aux nœuds et s'appuient principalement sur des architectures récurrentes ou attentionnelles pour capturer l'évolution temporelle à partir de la séquence des événements observés. Parmi eux, TGAT [?] adapte le mécanisme de GAT [?] au contexte temporel en intégrant un encodage du temps directement dans le mécanisme d'attention.

La plupart des GNN dynamiques reposent sur des stratégies d'échantillonnage multi-hop afin de récupérer un voisinage temporel étendu. Ce processus est explicite

dans la majorité des approches, mais peut également être implicite dans les modèles utilisant un RNN. CAWs [?] propose une alternative fondée sur des parcours causaux qui exploite l'hypothèse de fermeture triadique, selon laquelle deux nœuds partageant un voisin commun ont davantage de chances d'être connectés. Cette hypothèse est cependant moins adaptée à notre cas d'étude basé sur un graphe biparti.

Plus récemment, DyGFormer [?] simplifie cette approche en se limitant à un voisinage de distance 1, tout en ajoutant des informations de cooccurrence. Celles-ci sont calculées à partir de la fréquence d'apparition des nœuds dans les événements du voisinage temporel, puis injectées dans un Transformer. Les auteurs proposent également une implémentation unifiée facilitant la comparaison des modèles de graphes dynamiques. Comme DyGFormer, GraphMixer [?] se limite également à un voisinage de distance 1. Les auteurs critiquent l'utilisation systématique de RNN et de mécanismes d'attention dans les graphes dynamiques, en expliquant que ces architectures sont principalement nécessaires pour agréger efficacement des dépendances multi-hop. En se limitant à un voisinage local, GraphMixer remplace ces mécanismes par des architectures de type MLP-Mixer, beaucoup plus simples et rapides à inférer.

À l'inverse des modèles *Temporal Embedding*, les modèles *Temporal Neighborhood* maintiennent un état latent associé à chaque nœud déjà observé, mis à jour au fil des événements. Cette approche est plus coûteuse en mémoire et en calcul, mais elle permet de conserver un historique condensé des interactions passées et de fournir davantage de contexte aux RNN lors du traitement de nouvelles interactions. Du côté des graphes dynamiques continus avec mémoire, on retrouve notamment DyRep [?], qui distingue les événements de communication des événements d'association entre nœuds n'ayant jamais interagi auparavant. Cette approche est particulièrement adaptée aux multigraphes dynamiques, mais elle est moins pertinente dans notre cas où il n'existe qu'une seule arête possible par paire de nœuds. JODIE [?] est quant à lui conçu pour la prédiction de liens sur graphes bipartis. Le modèle maintient pour chaque utilisateur et chaque item un embedding dynamique mis à jour par des RNN mutuellement récurrents, en complément d'une représentation statique. Enfin, TGN [?] propose une formalisation générale des graphes dynamiques continus avec mémoire. Le modèle introduit une architecture modulaire composée d'un mécanisme de mémoire, d'une fonction de mise à jour, d'un module de message passing et d'un encodeur temporel. De nombreux modèles existants peuvent ainsi être interprétés comme des cas particuliers de ce formalisme.

Une vue d'ensemble de l'apprentissage sur graphes dynamiques est proposée par Feng et al. [?], qui présentent une synthèse mettant en évidence les différences conceptuelles entre les modèles dynamiques à temps discret et à temps continu, ainsi qu'un benchmark étendu des principales méthodes de ces deux catégories. Le tableau ?? fournit une comparaison synthétique de ces modèles selon leurs mécanismes d'attention, leur utilisation de RNN et de mémoire, ainsi que leur stratégie d'échantillonnage de voisinage.

Modèle	Attention	RNN	Mémoire	Echantillonnage
TGAT [?]	Oui	Non	Non	Multi-hop
CAWs [?]	Oui	Oui	Non	Parcours causal
DyGFormer [?]	Oui	Non	Non	1-hop
GraphMixer [?]	Non	Non	Non	1-hop
DyRep [?]	Non	Oui	Oui	Multi-hop
JODIE [?]	Non	Oui	Oui	Multi-hop
TGN [?]	Oui	Oui	Oui	Multi-hop

Table 3.2: Comparaison des principaux modèles de graphes dynamiques en temps continu

3.4.2.3 GNN non supervisé

L'ensemble des modèles précédemment énoncés sont conçus pour pouvoir être utilisés sur des tâches de prédiction de liens, qui relève de l'apprentissage auto-supervisé. Ce cadre est généralement considéré comme une sous-catégorie de l'apprentissage non supervisé, dans lequel des pseudo-labels sont construits automatiquement à partir des données brutes. Dans le cas des graphes, cela consiste à opposer des arêtes positives, correspondant aux interactions réellement observées dans le graphe, à des arêtes négatives, générées artificiellement à partir de paires de nœuds n'ayant jamais été connectées. Ces arêtes sont encodées puis passées dans une couche fully connected qui prédit la probabilité d'existence de l'arête. La prédiction obtenue est ensuite comparée à la vérité afin de calculer la fonction de perte. Cette méthode fonctionne mais la génération d'arcs négatifs pertinents constitue un problème difficile. Il est en effet, à la fois possible de produire des arcs négatifs qui pourraient en réalité être légitimes, ou au contraire de ne pas couvrir correctement la diversité des comportements anormaux lorsque ceux-ci sont échantillonnés de manière aléatoire. Ce problème est d'autant plus marqué dans des contextes tels que le nôtre, où les sous-graphes sont fortement structurés et contraints.

L'autre approche non supervisée couramment utilisée pour la détection d'anomalies consiste à formuler le problème comme une tâche de reconstruction des données. Pour se faire, on utilise un autoencodeur qui prend en entrée un graphe qui prend en entrée un graphe et génère une représentation latente compressée avant d'apprendre à reconstruire l'entrée d'origine à partir de cette représentation. Après avoir été entraîné sur des données ne contenant pas d'anomalies, l'autoencodeur est capable de reconstruire correctement les points considérés comme normaux, mais échoue davantage sur des données atypiques qu'il n'a jamais observées auparavant. L'erreur de reconstruction peut alors être utilisée comme score d'anomalie. Plus cette erreur est élevée, plus le point est susceptible d'être anormal. C'est notamment le principe utilisé par DONE [?], qui repose sur deux autoencodeurs parallèles. Le premier encode la structure des liens du graphe, tandis que l'autre encode les attributs des nœuds. Les auteurs proposent également AdONE [?], une extension

basée sur l'apprentissage adversarial permettant de construire des embeddings plus robustes aux valeurs aberrantes. GraphMAE[?] utilise également une architecture de type autoencodeur. Leur contribution principale consiste à appliquer un double masquage. Un masque est appliqué sur les attributs des nœuds avant l'encodage, puis un second masque est appliqué dans l'espace latent après l'encodage. Dans leur cas, l'encodeur est un GNN, tandis que le décodeur est un MLP utilisé. Cette stratégie de masquage vise à renforcer la robustesse des représentations apprises en forçant le modèle à inférer des informations manquantes à différents niveaux de représentation. STGMAE[?] est presque identique mais enrichit l'entrée du GNN par une couche d'encodage spatio-temporel. Toutefois, ces approches restent limitées au cadre des graphes statiques et ne prennent pas directement en compte l'évolution temporelle de la structure du graphe. DyGIS[?] adapte justement GraphMAE aux graphes discrets dynamiques mais, à notre connaissance, aucun travail de l'état de l'art ne propose encore d'étendre les méthodes de reconstruction au cas des graphes dynamiques en temps continu non supervisé.

Parmi les deux approches d'apprentissage non supervisées présentées, la prédiction d'arcs via l'auto-supervision semble plus facilement équilibrable, étant donné la nature binaire de la tâche. Cependant, la création d'arcs négatifs qui est nécessaire à l'auto-supervision est sujette à l'introduction de biais. En revanche, l'approche de reconstruction est moins susceptible à ces biais mais est plus difficile à guider.

3.4.3 Formalisation adapté à l'utilisation de GNN dynamiques

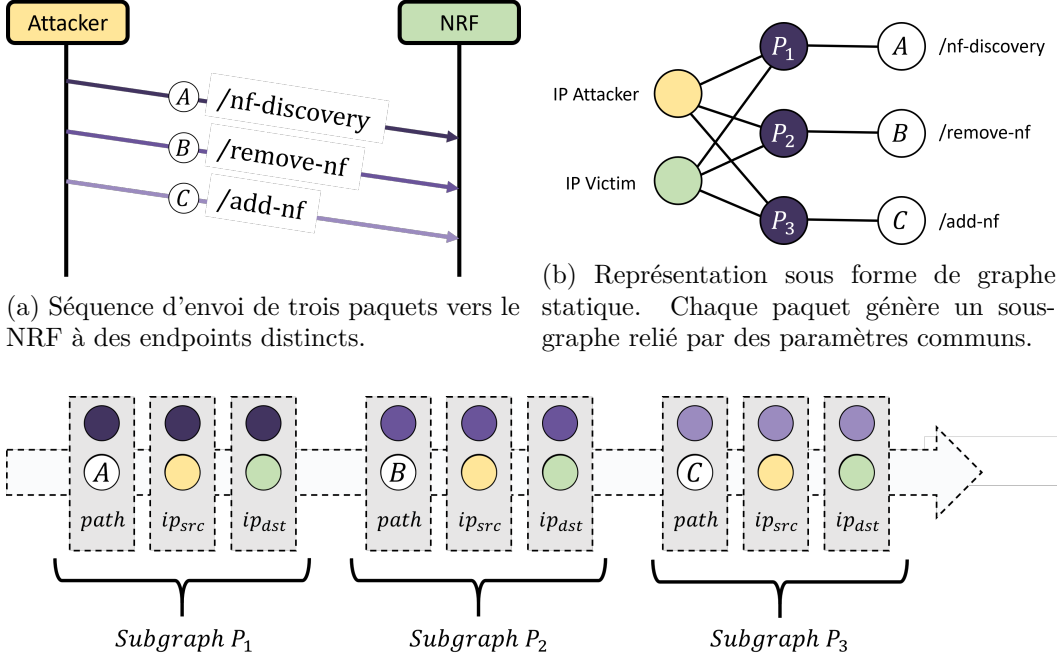
Pour considérer la dimension temporel et détecter les attaques d'inondation ou à plusieurs étapes, il faut intégrer l'information temporel à notre détection et donc considérer les graphes dynamiques. On pourrait utiliser les graphes dynamiques discrets et c'est ce que fait d'ailleurs GSAD [?]. Cette façon de gérer les graphes dynamiques est facile à comprendre et à implémenter, car les captures à intervalle régulier de l'état du graph forment une séquence de graphes indépendants sur lesquels on peut appliquer les GNN traditionnels sans modification majeure. Leur principal inconvénient est qu'ils sont très lourds et peu efficaces pour détecter les dépendances sur de longues périodes.

Nous nous intéressons donc davantage aux graphes dynamiques continus, tels qu'introduits dans le cadre de TGN [?]. Dans cette approche, plutôt que de représenter le graphe comme un objet statique décrivant un état à un instant donné, celui-ci est modélisé comme une séquence d'événements qui le construit progressivement.

$$\mathcal{G} = \{e_1, e_2, \dots, e_n\}$$

où e_i désigne l'événement apparaissant à la i -ème position dans la séquence.

Ces événements correspondent généralement à des ajouts, suppressions ou modifications de nœuds et d'arêtes. Toutefois, notre cadre est plus simple car



(c) Passage vers un graphe dynamique continu, où chaque arête est modélisée comme un événement contenant les informations des nœuds et arcs concernés.

Figure 3.8: Différentes représentations d'une même séquence d'échanges entre un attaquant et le NRF : diagramme de séquence, graphe statique, puis graphe dynamique continu.

notre graphe est purement additif. Chaque arête qui serait ajoutée dans un graphe traditionnel est représentée comme un événement défini par cette arête ainsi que par les deux nœuds qu'elle relie.

$$e_i = (u, v_i, x_i),$$

où $u \in \mathcal{V}_c$ désigne un nœud central, $v_i \in \mathcal{V}_f$ un nœud de paramètre contenant la valeur de ce dernier et $\mathbf{z}_i \in \mathbb{R}^d$ le nom du paramètre contenu dans l'arête.

Cette formulation nécessite d'adapter les algorithmes classiques utilisés dans les GNN, mais elle permet une représentation beaucoup plus fine, dans laquelle aucune information temporelle n'est perdue. La figure ?? illustre la transformation d'une forme de graphe statique à celle d'un graphe dynamique continu.

3.4.4 Fonctionnement détaillé des GNN dynamique

Dans leurs travaux, Rossi et al. [?] proposent un cadre de formalisation des GNN dynamiques, permettant de représenter les travaux de l'état de l'art comme des spécifications de leur modèle. Nous nous appuyons sur ce framework pour présenter et comprendre le fonctionnement d'un GNN dynamique. La figure ?? illustre les quatre modules principaux définis par TGN [?] : un module de mémoire,

un module d'agrégation de messages, un module de mise à jour de la mémoire et un module d'inférence.

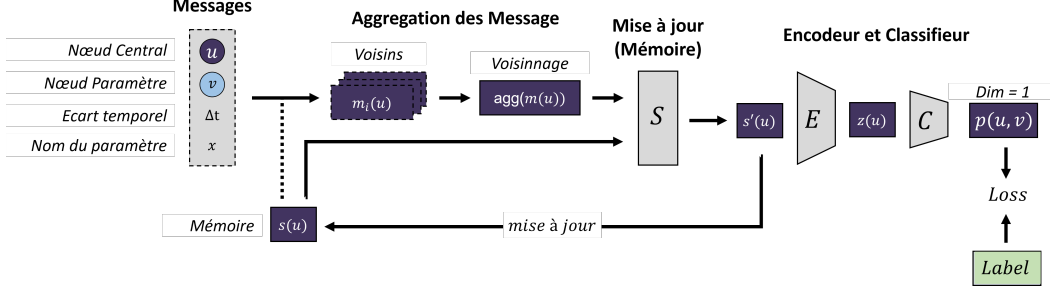


Figure 3.9: Vue d'ensemble de l'architecture générale d'un GNN Dynamique Continu, illustrant les quatre modules principaux : mémoire, agrégation de messages, mise à jour de la mémoire et inférence.

3.4.4.1 Création des messages

La première étape d'un GNN consiste à construire un message contenant l'ensemble des informations associées à un événement du graphe dynamique.

$$m_i = msg(s(u), s(v_i), x_i, \Delta t)$$

Les vecteurs associés aux nœuds centraux $s(u)$ et aux nœuds paramètres $s(v_i)$ sont initialisés comme des vecteurs nuls. Si ces nœuds ont déjà été impliqués dans des événements précédents, leurs représentations déjà calculées est alors réutilisées. Le vecteur associé à l'arête x_i est quant à lui déterminé à partir du nom du paramètre porté par l'événement. Dans notre cas, ce nom de paramètre correspond à un token textuel et il est donc nécessaire de l'encoder au préalable sous forme d'un vecteur numérique. La méthode utilisée pour cet encodage est présentée dans la section suivante. Il existe diverses méthodes pour définir l'intervalle temporel Δt mais la méthode la plus fréquemment employée consiste à calculer la différence temporelle entre l'échantillon et le dernier paquet encodé. Ce dernier est représenté sous forme de vecteur de dimensions identiques aux autres composantes du système. Un paramètre ajustable permet alors de moduler la granularité de cette mesure.

3.4.4.2 Agrégation des messages

L'étape d'agrégation consiste à considérer l'ensemble des voisins du nœud source en cours de traitement, c'est-à-dire tous les événements m_i précèdent un temps t dans lesquels ce nœud apparaît. On obtient ainsi une collection de messages qu'on d'agrége ensuite afin de produire un vecteur unique représentant le voisinage de distance 1 du nœud considéré

$$\bar{m}_i(t) = \text{agg}(m_i(t_1), \dots, m_i(t))$$

Cette agrégation peut être réalisée de différentes manières. TGN [?] propose plusieurs stratégies, parmi lesquelles l'utilisation de RNN, d'un mécanisme d'attention, une moyenne des messages ou bien de simplement conserver l'événement le plus récent.

3.4.4.3 Mise à jour de la mémoire

Afin de prendre en compte la dimension temporelle, les modèles dynamiques de type *Temporal Neighborhood* disposent d'un module de mémoire, absent des modèles *Temporal Embedding*. Ce module combine le vecteur agrégé avec l'historique des représentations précédentes des éléments constitutifs du message. Cet ensemble est ensuite transmis à un RNN, qui joue le rôle de module de mémoire apprenable. Ce module produit un nouveau vecteur représentant à la fois le message et son historique.

$$s(u) = mem(x_i, s(u))$$

Cette nouvelle représentation est utilisée pour les modules futurs et remplace également l'ancienne valeur stockée en mémoire. TGN [?] propose notamment d'utiliser des architectures de type LSTM ou GRU pour implémenter ce module de mémoire.

3.4.4.4 Encodeur et Classifieur

L'encodeur est le module chargé de calculer la représentation des nœuds à partir de leur mémoire et des interactions temporelles. Plusieurs variantes peuvent être utilisées :

- **Identity (id)** : l'encodage correspond directement à la mémoire du nœud.
- **Time projection (time)** : la mémoire est modulée en fonction du temps écoulé depuis la dernière interaction, comme dans JODIE [?].
- **Temporal Graph Attention (attn)** : plusieurs couches de graph attention agrègent l'information provenant du voisinage temporel du nœud. À chaque couche, les voisins temporels sont récupérés puis combinés via un mécanisme de multi-head attention.
- **Temporal Graph Sum (sum)** : version plus simple et rapide où les informations des voisins temporels sont agrégées par somme puis projetées linéairement.

Pour les modèles *attn* et *sum*, un encodage temporel de type *Time2Vec* est utilisé afin de représenter les écarts temporels entre interactions. Ces méthodes permettent également de limiter le problème de *staleness* en mettant à jour les représentations grâce aux informations des voisins récents.

3.4.4.5 Conclusion

Nous avons présenté dans cette section le fonctionnement détaillé des GNN Dynamiques. Toutefois, il s'agit d'un cadre de fonctionnement général, alors que notre problématique est plus spécifique et nécessite la mise en œuvre de plusieurs ajustements méthodologiques.

3.4.5 Adaptation à notre cas d'utilisation

Pour mieux répondre à notre problématique, nous proposons deux ajustements justifiés par notre cadre d'utilisation. Dans un premier temps, nous supprimons l'information temporelle explicite des messages. Dans un second temps, nous adaptons le mécanisme d'agrégation standard en une version spécifique, conçue pour notre structure bipartite.

Le premier ajustement consiste à supprimer l'information temporelle explicite du modèle. Concrètement, cela se traduit par le retrait du terme Δt dans la formulation des messages. Dans notre contexte, l'information temporelle est déjà implicitement capturée par les séquences d'interactions lorsqu'elle est traitée par le RNN. La mention explicite du terme Δt est principalement pertinente pour identifier d'éventuelles pauses ou délais dans le trafic. Toutefois, cette information ne constitue pas un indicateur suffisamment représentatif d'une attaque et induit souvent, au contraire, des biais de surapprentissage. Pour cette raison, nous avons choisi de retirer cette composante temporelle de la formulation des messages.

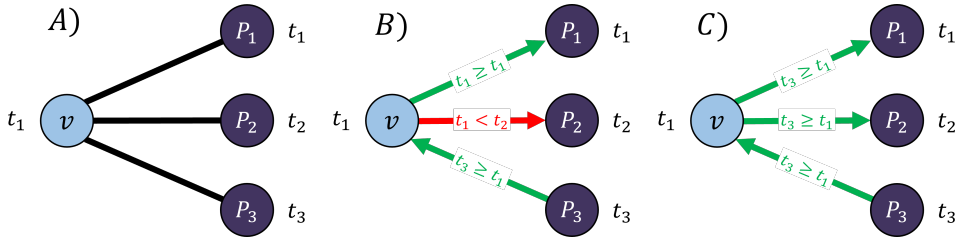


Figure 3.10: Représentation du problème lié au parcours multi saut dans notre graphe biparti. La figure illustre (A) le graphe avec les arrivées temporelles des paquets, (B) la méthode classique de propagation multi-sauts contrainte temporellement, et (C) notre approche modifiée permettant de récupérer correctement les dépendances temporelles en propageant le temps du nœud initial.

Le deuxième ajustement concerne la construction des voisinages multi-sauts. Dans un GNN dynamique classique, un nœud ne peut agréger que des voisins apparus avant lui. Cependant, dans notre graphe bipartite, les nœuds paramètres sont réutilisés et conservent leur temps de première apparition. Cela crée un problème lors de l'exploration multi-sauts. Certains nœuds centraux, pourtant antérieurs au nœud cible ne sont pas récupérés, car ils sont postérieurs au nœud paramètre intermédiaire. Pour résoudre ce problème, nous propageons le temps d'arrivée du nœud cible lors de l'exploration du voisinage, plutôt que celui des

noeuds intermédiaires. Cela permet de conserver un voisinage temporel cohérent malgré la réutilisation des noeuds paramètres. Le problème est ainsi que la solution que nous proposons sont représentés sur la figure ??.

Ces ajustements répondent directement aux spécificités de notre cas d’usage et ont donc été intégrés à notre approche. La suppression de l’information temporelle explicite permet de limiter les risques de surapprentissage liés aux irrégularités du trafic, tandis que la modification du mécanisme de propagation garantit une exploration multi-sauts cohérente dans le contexte particulier de notre graphe biparti.

3.4.5.1 Problèmes ouverts

D'autres pistes d'ajustement ont également été envisagées au cours de ce travail, mais n'ont finalement pas été intégrées à l'approche proposée. Elles restent néanmoins intéressantes à mentionner, car leur considération est légitime et soulève plusieurs questions pertinentes quant à notre formalisation.

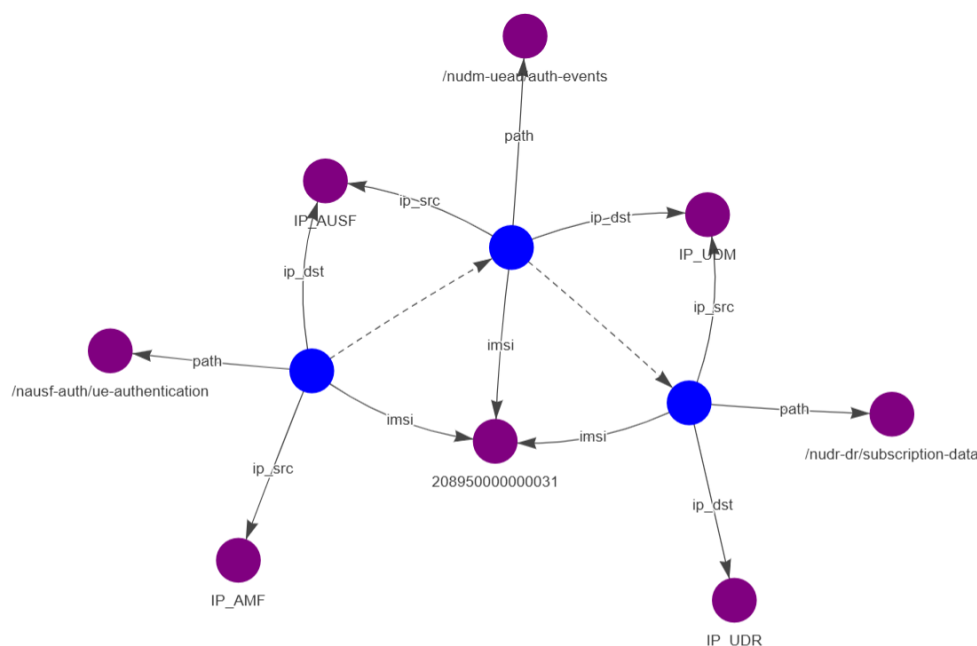


Figure 3.11: Représentation sous forme de graphe d'une procédure composée de trois paquets. Les arcs orientés en pointillés représentent un nouveau type de liaison entre noeuds central permettant de représenter l'ordre temporel des échanges.

Dans un réseau 5G, le déclenchement d’une procédure engendre la génération de séquences ordonnées de messages. Il peut être envisagé de représenter explicitement ces séquences ainsi que leur ordre au sein du graphe au moyen d’une formalisation dédiée. Une approche consiste à orienter le graphe et à introduire un type de liaison spécifique permettant de relier un nœud central à son successeur dans la séquence. La Figure ?? illustre cette représentation, qui présente l’avantage d’être aisément

interprétable par un lecteur humain tout en rendant explicites les séquences et leur ordre d'exécution. Cependant, cette approche n'apporte pas d'information supplémentaire au modèle, dans la mesure où la structure séquentielle est déjà encodée via les graphes dynamiques continus. Néanmoins, elle conserve un intérêt pour l'analyse humaine et peut ainsi être ajoutée a posteriori de l'inférence afin d'améliorer la lisibilité des résultats.

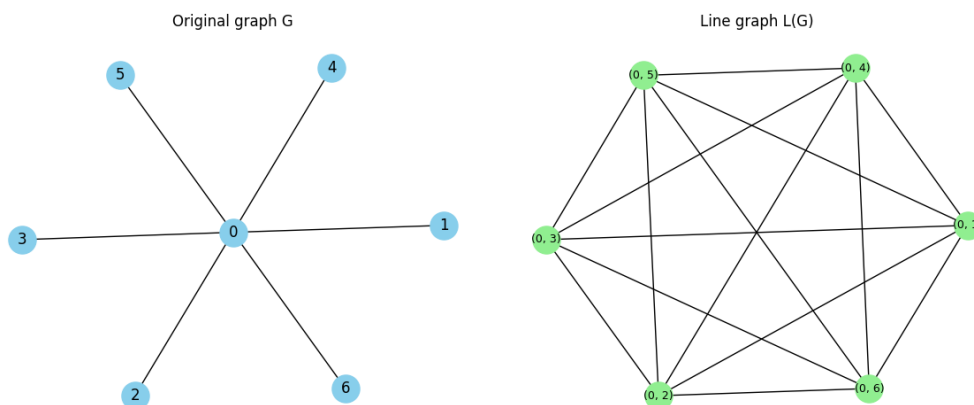


Figure 3.12: Illustration de la transformation d'un graphe classique (à gauche) en *line graph* (à droite). Dans cette représentation, les arêtes du graphe initial deviennent les nœuds du *line graph*, tandis que les relations entre arêtes sont représentées par de nouvelles connexions.

EGAT[?] propose de donner plus d'importance à l'information portée par les arêtes en réalisant une agrégation sur les arêtes au lieu de sur les nœuds comme classiquement. Pour se faire ils convertissent leur graphe en *line graph* où les nœuds correspondent aux arêtes du graphe original, tandis que ses arêtes traduisent les relations entre ces dernières. La Figure ?? illustre cette transformation. Néanmoins, dans notre cas d'étude, les graphes considérés sont fortement connectés et comportent un grand nombre d'arêtes. Or, la construction du *line graph* entraîne une augmentation significative de la complexité structurelle, puisque chaque arête du graphe initial devient un nœud supplémentaire. Cette transformation ne passe donc pas à l'échelle dans notre configuration et conduit rapidement à une explosion de la complexité computationnelle et mémoire.

Ces différentes approches montrent que plusieurs extensions de la formalisation initiale sont envisageables. Néanmoins, bien qu'elles présentent des avantages évidents, elles soulèvent des problèmes structurels et computationnels qui empêchent leur intégration à notre formalisation.

3.5 Encodage des paramètres de dimension non-fini (NLP)

3.5.1 Sélection et encodage des features

3.6 Conclusion